

UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



PROYECTO

Segunda Entrega de Proyecto

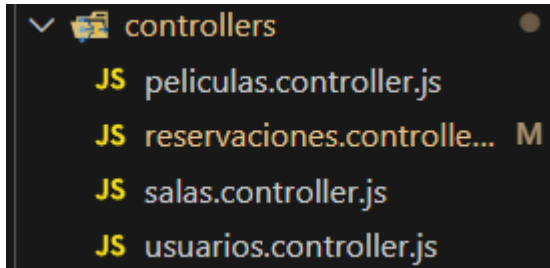
AUTOR

Sajvin Gómez, Mario Daniel (1612921)

Enlace a Repositorio:

<https://github.com/DanielSajvin/cine-app/tree/develop>

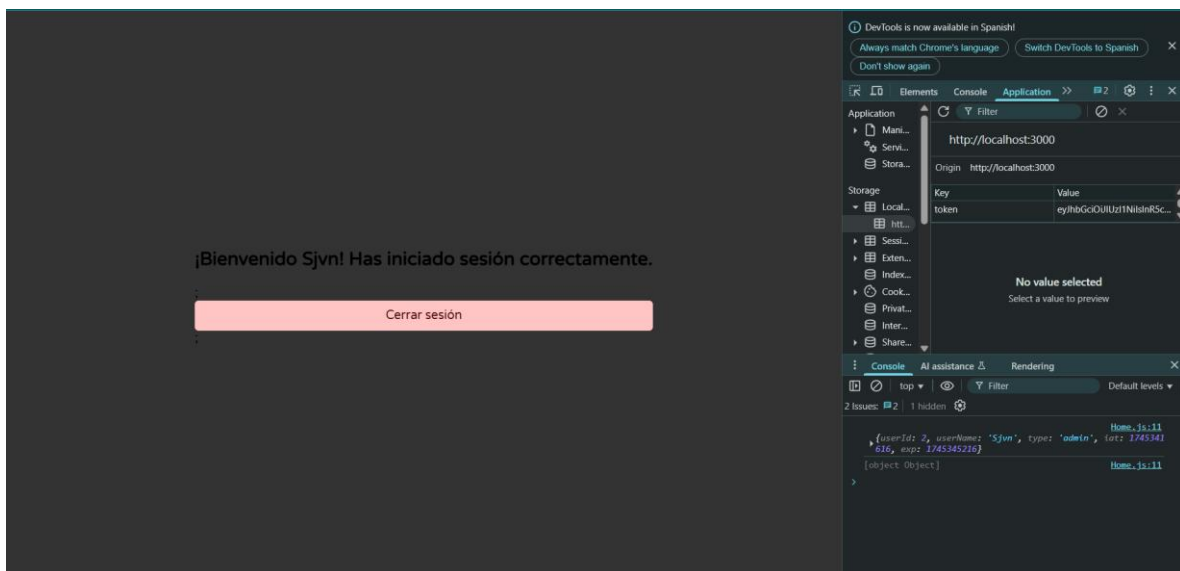
Avances Backend



CRUD completo de películas, reservaciones, salas, usuario.

Las acciones que cada usuario pueda realizar van a depender de los permisos que tengan, es decir, que existen Endpoints protegidos donde se revisa si el usuario es Administrador o no, como también se revisa el JWT.

El JWT se crea al momento de iniciar sesión.



Por ejemplo, en la parte de salas se tienen los Endpoints

```
const express = require("express");
const salasRouter = express.Router();
```

```

const { verificarToken, verificarAdmin } =
require("../middlewares/auth");

const {
  crearSala,
  actualizarSala,
  listarSalas,
  eliminarSala,
} = require("../controllers/salas.controller");

// Definir la ruta
salasRouter.post("/crearSala", verificarToken, verificarAdmin, crearSala);
// c
salasRouter.get("/listarSalas", verificarToken, verificarAdmin,
listarSalas); // r
salasRouter.put("/actualizarSala/:id", verificarToken, verificarAdmin,
actualizarSala); // u
salasRouter.delete("/eliminarSala/:id", verificarToken, verificarAdmin,
eliminarSala); // d

module.exports = salasRouter;

```

Entonces podemos ver que cada Endpoint se verifica el JWT y por ejemplo para eliminar una sala también se verifica si es Administrador o no para poder completar la acción.

Toda esta verificación la realiza el middleware “auth.js”

```

const jwt = require("jsonwebtoken");

const verificarAdmin = (req, res, next) => {
  console.log("Usuario autenticado: ", req.user);
  if (req.user.type !== "admin") {
    return res
      .status(403)
      .json({
        mensaje: "Acceso denegado. Se requiere el rol de administrador",
      });
  }
  next();
};

const verificarToken = (req, res, next) => {

```

```

    const token = req.headers.authorization?.split(" ")[1]; // obtener el token desde el header

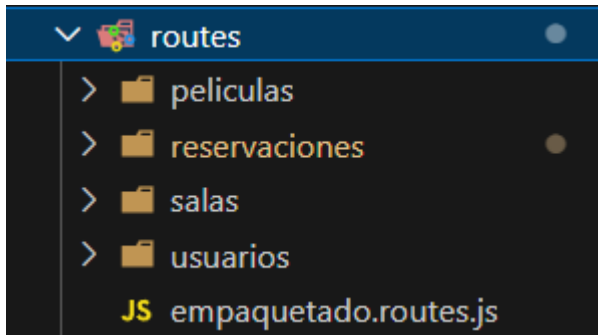
    if (!token) {
        return res.status(401).json({ mensaje: "No hay token" });
    }

    try {
        const decoded = jwt.verify(token, process.env.SECRET_KEY);
        req.user = decoded; // guardar el usuario decodificado en el request
        console.log("Token decodificado:", decoded);
        next();
    } catch (error) {
        return res.status(403).json({ mensaje: "Token inválido o expirado" });
    }
};

module.exports = { verificarAdmin, verificarToken };

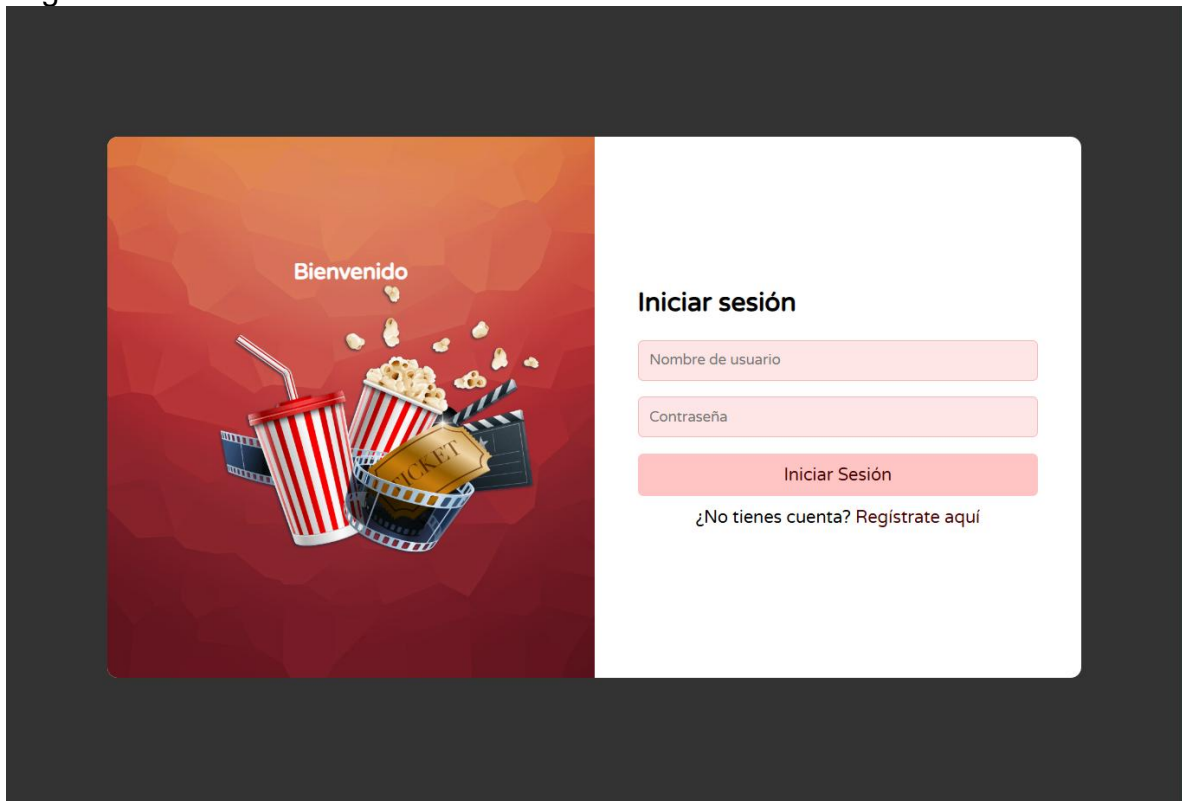
```

Las rutas se encuentran estructuradas de la siguiente manera:

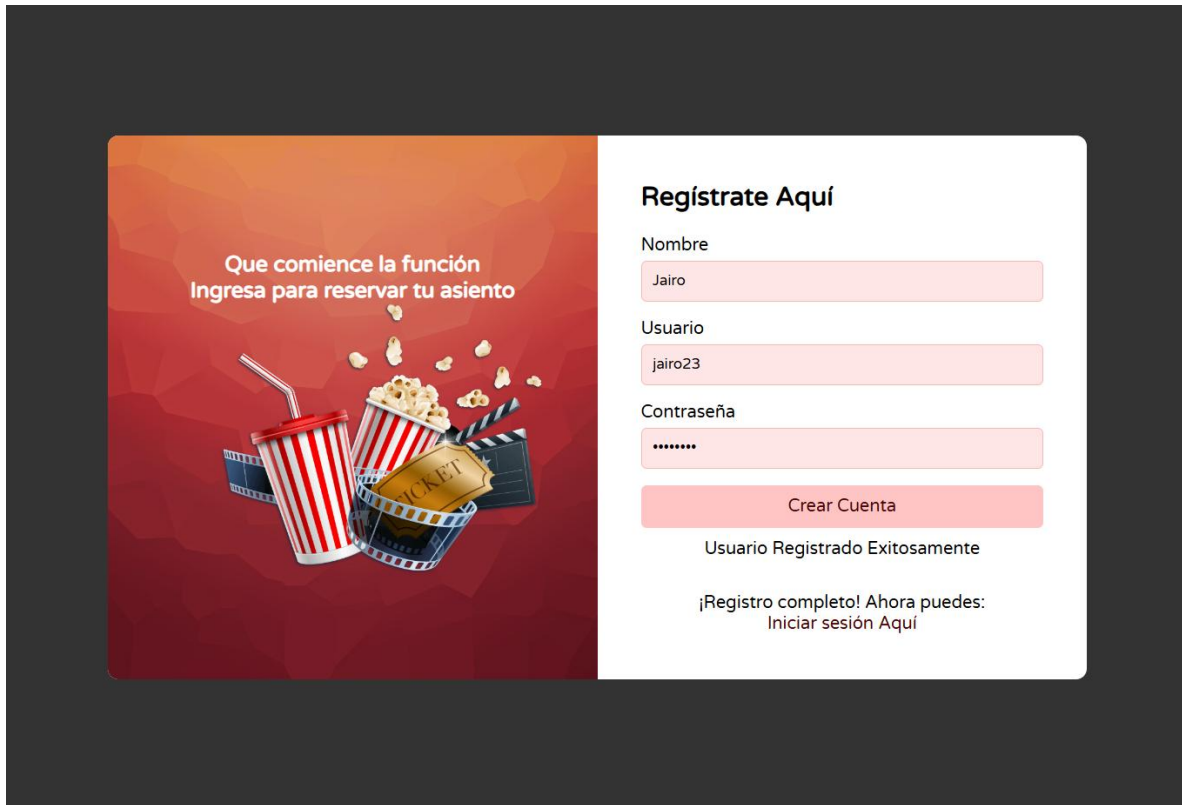


Avances Frontend

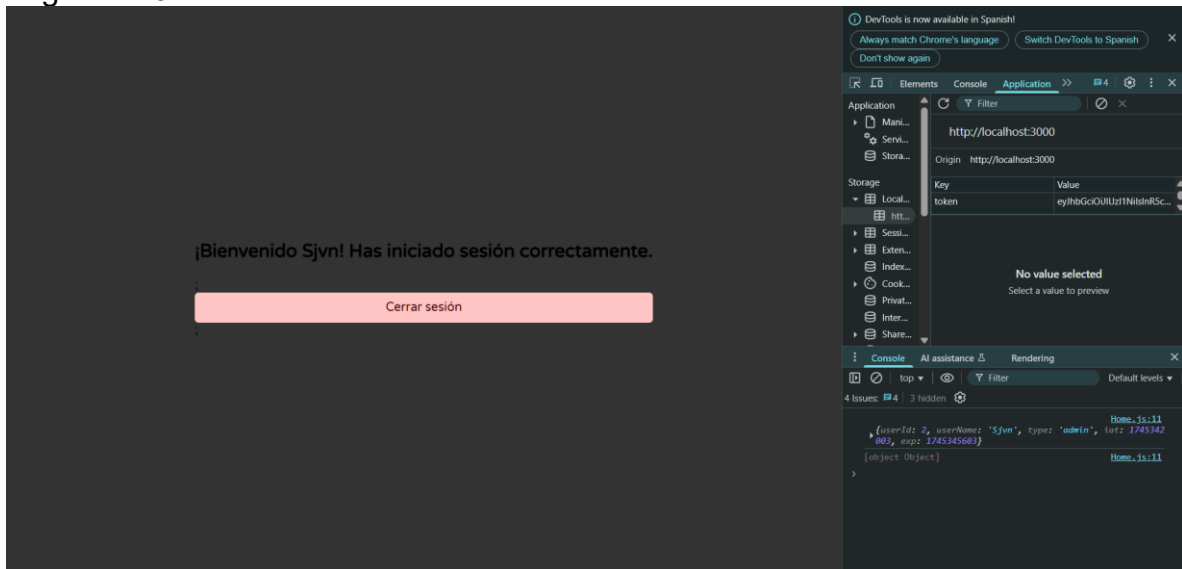
Login Funcional



Register Funcional



Login con JWT



De momento solo se tiene la vista Home después del Login, pero este se encuentra protegido de la siguiente manera:

En el componente RutaPrivada se verifica el Token y de lo contrario se le redirige al Login:

```
import React from "react";
import { Navigate } from "react-router-dom";

const RutaPrivada = ({ children }) => {
  const token = localStorage.getItem("token");

  if (!token) {
    return <Navigate to="/login" />;
  }

  return children;
};

export default RutaPrivada;
```

El componente Login se encuentra de la siguiente manera:

```
import { Link } from 'react-router-dom';
import Formulario from '../formulario/Formulario';
import './login.css';

function Login() {
  return (
    <div class="container">
      <div class="login-box">
        <h2>Inicia Sesión</h2>
        <Formulario />
        <p>
          <Link to="/register">>¿No tienes una cuenta? Regístrate
Aquí</Link>
        </p>
      </div>
      <div class="image-section">
        <div class="overlay">
          <p>
            Que comience la función<br></br>Ingresa para reservar tu asiento
          </p>
        </div>
      </div>
    </div>
  );
};
```

```
}  
  
export default Login;
```

Pero donde sucede la mayor parte de la lógica es en el componente Formulario

```
import React, { useState } from "react";  
import { useNavigate, Link } from "react-router-dom";  
import "../login.css";  
  
function Formulario() {  
  const [userName, setUserName] = useState("");  
  const [password, setPassword] = useState("");  
  const [mensaje, setMensaje] = useState("");  
  
  const navigate = useNavigate();  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
  
    const datos = { userName, password };  
  
    try {  
      const res = await fetch(  
        "http://localhost:4000/api/usuarios/loginUsuario",  
        {  
          method: "POST",  
          headers: {  
            "Content-Type": "application/json",  
          },  
          body: JSON.stringify(datos),  
        },  
      );  
  
      const data = await res.json();  
  
      if (res.ok) {  
        localStorage.setItem("token", data.token);  
        setMensaje("Inicio de sesión exitoso. Redirigiendo...");  
        setTimeout(() => {  
          navigate("/home"); // Redirige a la página de inicio después de 2 segundos  
        }, 1500);  
      } else {  
        setMensaje("Error al iniciar sesión. Verifica tus credenciales.");  
      }  
    } catch (error) {  
      console.error("Error al iniciar sesión:", error);  
      setMensaje("Error al iniciar sesión. Verifica tus credenciales.");  
    }  
  }  
}
```



```

        setMensaje(data.mensaje);
    }
} catch (error) {
    setMensaje("Error al conectar con el servidor. Inténtalo de nuevo.");
    console.error("Error:", error);
}
};

return (
    <div className="container">
        <div className="image-section">
            <div className="overlay">Bienvenido</div>
        </div>

        <div className="login-box">
            <h2>Iniciar sesión</h2>
            <form onSubmit={handleSubmit}>
                <input
                    type="text"
                    placeholder="Nombre de usuario"
                    value={userName}
                    onChange={(e) => setUsername(e.target.value)}
                    required
                />

                <input
                    type="password"
                    placeholder="Contraseña"
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    required
                />

                <button type="submit">Iniciar Sesión</button>
            </form>

            {mensaje && <p>{mensaje}</p>}}

            <p>
                ¿No tienes cuenta?{" "}
                <Link to="/register" className="login-link">
                    Regístrate aquí
                </Link>
            </p>
        </div>
    </div>

```

```
    </div>  
  );  
}  
  
export default Formulario;
```

Para el register funciona de la misma manera, el Formulario es el que hace la mayor parte de la lógica .